# Fissure

Active Web-Application Vulnerability Scanner

**Web & Application Security (WAS) Lab**
**Mini Project**

## Problem Statement

Passive and reconnaissance scanners report missing headers, open ports and known-file exposure, but they never confirm whether an input is **actually exploitable**. That leaves a gap between "looks risky" and "is vulnerable." **Fissure** closes it: given a single **authorized** target URL, it crawls the web application, automatically discovers every query parameter and form field, and **actively injects payloads** to confirm real, exploitable vulnerabilities — then produces a scored, shareable report. It is built for authorized lab use against deliberately vulnerable targets such as DVWA.

**Project scope.** **Step 1 — Fissure** (this submission): the complete deterministic scanner below. **Step 2 — Fissure AI** (planned): a Claude-powered agent that drives Fissure as its toolbelt, chaining and confirming findings autonomously.

## How It Works — one URL in, a scored report out

**Scope guard** → **Crawl** → **Active tests** → **Discovery** → **Report**

One scan pipeline: check the target is in scope → crawl from the URL to harvest injectable inputs → fire active payloads at each input → passively audit headers → probe for sensitive files → score and render JSON + HTML + SARIF. Modules: **crawler, sqli, cmdi, xss, traversal, ssti, cors, redirect, discovery, headers**.

## Detection Capabilities

| Vulnerability class | Techniques used | How a hit is confirmed |
|---|---|---|
| **SQL Injection** | Error- · Boolean- · Time-based blind | DB error · true/false size diff · ~5s delay |
| **Command Injection** | Time-based blind (sleep/ping) | Response delayed ~6s by injected shell command |
| **Reflected XSS** | Markers in HTML / attribute / script | Payload reflected unescaped in the response |
| **Path Traversal / LFI** | ../ sequences + encodings | Response contains /etc/passwd or win.ini contents |
| **Template Injection (SSTI)** | {{a*b}} / ${} / <%=%> marker probe | Engine evaluates the arithmetic expression |
| **CORS misconfiguration** | Reflected Origin probe | ACAO echoes attacker origin with credentials |
| **Open Redirect** | External canary on nav params | 3xx Location or meta-refresh points off-site |
| **Default / sensitive files** | Path wordlist + autoindex check | HTTP 200 on sensitive path · 'Index of /' |
| **Security headers / cookies** | Passive headers & Set-Cookie flags | Missing CSP/HSTS/XFO · HttpOnly/Secure/SameSite |

## How Fissure Compares to Industry Tools

| Tool | Category | Confirms exploitability | Footprint |
|---|---|---|---|
| **OWASP ZAP** | Full DAST (active + passive) | Yes — broad | Large · Java daemon/GUI |
| **Burp Suite** | Intercepting proxy + scanner | Yes (Pro, paid) | Large · commercial |
| **Nikto** | Server / misconfig scanner | No — signature matches | Small · Perl CLI |
| **sqlmap** | SQL-injection exploitation | Yes — SQLi only | Medium · CLI |
| **Fissure (ours)** | **Focused active scanner + GUI** | **Yes — SQLi/XSS/cmdi/LFI** | **Tiny · single .exe** |

## Why Fissure

- **Confirms, not just flags** — it injects and observes real behaviour (DB error, 5s delay, unescaped reflection), so a finding means *exploitable*, not merely "looks odd."
- **Right-sized & transparent** — one small, readable Python codebase vs. heavyweight Java/commercial suites; every check is auditable, not a black box.
- **Safe-by-default** — refuses public targets without explicit authorization; payloads are non-destructive (sleep/ping, never rm).
- **Zero-friction demo** — a single **Windows .exe** (no Python needed) plus a bundled vulnerable target; 38 automated tests; verified live against DVWA.
- **Honest severity** — context-aware CLEAN→CRITICAL score with CWE ids; correctly reports *nothing* when a target is properly fixed.
- **Built to grow** — a clean module/engine/report contract that becomes the toolbelt for **Fissure AI** (Step 2, a Claude agent that drives it).

Verified end-to-end against DVWA running in Docker. Fissure confirmed **error-based SQL injection**, **time-based blind SQLi** (5-second delay) and **reflected XSS** at low/medium security. Crucially, on **high-security SQLi** — where DVWA uses a parameterized query — it correctly reports **nothing**, demonstrating that it confirms true exploitability rather than pattern-matching. Benchmarked on a controlled target: **precision / recall / F1 = 1.00** (9/9, 0 false positives). 38 automated tests pass.

**Tech stack:** Python 3.10+ · httpx (async) · Tkinter GUI · PyInstaller · MIT licensed. **Deliverables:** source package, CLI, standalone Fissure.exe, JSON + HTML reports, automated tests, bundled VulnLab demo target.